PostgreSQL Health Report Function Package

health_report_function.sql

```
-- health_report_function.sql
-- PostgreSQL Database Health Report "package"
-- Creates:
--   * public.health_alerts table (history of alerts per run)
--   * public.run_health_report()    -> detailed checks + persists alerts
--   * public.run_health_summary()   -> roll-up summary for latest run
--   * public.run_health_critical()  -> CRITICAL-only for latest run
--   * public.has_critical_alerts()  -> boolean + counts for latest run
--
-- Usage:
--   SELECT * FROM run_health_report();
--   SELECT * FROM run_health_summary();
--   SELECT * FROM run_health_critical();
--   SELECT * FROM has_critical_alerts();
--   SELECT * FROM health_alerts WHERE severity='CRITICAL';
--
-- Notes:
-- * This is "inside-the-database" telemetry. It cannot see OS free disk or kernel metrics.
-- * Thresholds are conservative defaults; adjust in the "thresholds" CTE in run_health_report().
-- * Designed to run on Postgres 12+ (best on 13+). Some metrics depend on stats being enabled.

BEGIN;

-- ---------- 1) Storage for alert history ----------
CREATE TABLE IF NOT EXISTS public.health_alerts (
  id          bigserial PRIMARY KEY,
  run_id      uuid            NOT NULL,
  observed_at  timestamptz     NOT NULL DEFAULT now(),
  severity     text            NOT NULL CHECK (severity IN ('CRITICAL','WARN','INFO')),
  category     text            NOT NULL,
  check_name   text            NOT NULL,
  status       text            NOT NULL CHECK (status IN ('OK','ALERT')),
  message      text            NOT NULL,
  details      jsonb           NOT NULL DEFAULT '{}'::jsonb
);

CREATE INDEX IF NOT EXISTS health_alerts_run_id_idx
  ON public.health_alerts (run_id);

CREATE INDEX IF NOT EXISTS health_alerts_severity_idx
  ON public.health_alerts (severity);

CREATE INDEX IF NOT EXISTS health_alerts_observed_at_idx
  ON public.health_alerts (observed_at);

COMMENT ON TABLE public.health_alerts IS
'Health report alerts. Each run of run_health_report() generates a run_id and stores WARN/INFO/CRITICAL

-- ---------- 2) Helper: normalize severity ----------
CREATE OR REPLACE FUNCTION public._health_severity_rank(p_severity text)
RETURNS int
LANGUAGE sql
IMMUTABLE
AS $$
  SELECT CASE upper(p_severity)
           WHEN 'CRITICAL' THEN 3
           WHEN 'WARN'     THEN 2
           WHEN 'INFO'     THEN 1
           ELSE 0
         END;
$$;

-- ---------- 3) Main: run_health_report() ----------
-- Returns the full report AND persists alerts in health_alerts (one row per check outcome).
CREATE OR REPLACE FUNCTION public.run_health_report()
```

```
RETURNS TABLE (
  run_id      uuid,
  observed_at timestamptz,
  severity    text,
  category    text,
  check_name  text,
  status      text,
  message     text,
  details     jsonb
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_run_id uuid := NULL;
  v_now    timestamptz := now();
BEGIN
  -- Ensure we have a UUID generator
  BEGIN
    v_run_id := gen_random_uuid();
  EXCEPTION WHEN undefined_function THEN
    BEGIN
      EXECUTE 'CREATE EXTENSION IF NOT EXISTS pgcrypto';
      v_run_id := gen_random_uuid();
    EXCEPTION WHEN insufficient_privilege THEN
      BEGIN
        EXECUTE 'CREATE EXTENSION IF NOT EXISTS "uuid-ossp"';
        EXECUTE 'SELECT uuid_generate_v4()' INTO v_run_id;
      EXCEPTION WHEN OTHERS THEN
        v_run_id := (
          substr(md5(random()::text || clock_timestamp()::text), 1, 8) || '-' ||
          substr(md5(random()::text || clock_timestamp()::text), 9, 4) || '-' ||
          substr(md5(random()::text || clock_timestamp()::text), 13, 4) || '-' ||
          substr(md5(random()::text || clock_timestamp()::text), 17, 4) || '-' ||
          substr(md5(random()::text || clock_timestamp()::text), 21, 12)
        )::uuid;
      END;
    END;
  END;

  WITH report AS (
    WITH
    thresholds AS (
      -- Tune thresholds here.
      SELECT
        0.90::numeric AS conn_critical_pct,
        0.75::numeric AS conn_warn_pct,
        interval '30 minutes' AS long_query_critical,
        interval '5 minutes'  AS long_query_warn,
        interval '5 minutes'  AS lock_wait_critical,
        interval '1 minute'   AS lock_wait_warn,
        20::int AS waiting_locks_critical,
        5::int  AS waiting_locks_warn,
        0.90::numeric AS cache_hit_critical_below,
        0.95::numeric AS cache_hit_warn_below,
        0.20::numeric AS dead_tuple_ratio_critical,
        0.10::numeric AS dead_tuple_ratio_warn,
        1024^3::bigint AS unused_index_info_min_bytes,       -- 1 GB
        100*1024^2::bigint AS unused_index_warn_min_bytes,   -- 100 MB
        interval '5 minutes' AS repl_lag_warn,
        interval '15 minutes' AS repl_lag_critical
    ),
    basics AS (
      SELECT
        v_run_id::uuid AS run_id,
        v_now::timestamptz AS observed_at,
        current_database() AS dbname,
        version() AS server_version,
        pg_postmaster_start_time() AS postmaster_start_time,
        now() - pg_postmaster_start_time() AS uptime,
        pg_is_in_recovery() AS is_standby
    ),
    settings AS (
```

```
  SELECT
    current_setting('max_connections')::int AS max_connections,
    current_setting('autovacuum_freeze_max_age')::bigint AS freeze_max_age
),
db_stats AS (
  SELECT
    d.datname,
    d.numbackends,
    d.blks_read, d.blks_hit,
    d.temp_bytes,
    d.deadlocks,
    d.stats_reset,
    pg_database_size(d.datname) AS db_size_bytes
  FROM pg_stat_database d
  WHERE d.datname = current_database()
),
conn_usage AS (
  SELECT
    s.max_connections,
    ds.numbackends,
    (ds.numbackends::numeric / NULLIF(s.max_connections,0)) AS pct_used
  FROM settings s CROSS JOIN db_stats ds
),
activity_long AS (
  SELECT
    pid, usename, application_name, client_addr,
    now() - query_start AS runtime,
    left(regexp_replace(query, '\s+', ' ', 'g'), 400) AS query_sample
  FROM pg_stat_activity
  WHERE state = 'active'
    AND query_start IS NOT NULL
    AND (now() - query_start) > (SELECT long_query_warn FROM thresholds)
    AND pid <> pg_backend_pid()
),
waiting_locks AS (
  SELECT
    a.pid, a.usename, a.application_name, a.client_addr,
    now() - a.query_start AS waiting_for,
    left(regexp_replace(a.query, '\s+', ' ', 'g'), 400) AS query_sample
  FROM pg_stat_activity a
  WHERE a.wait_event_type = 'Lock'
    AND a.query_start IS NOT NULL
    AND pid <> pg_backend_pid()
),
lock_counts AS (
  SELECT count(*)::int AS waiting_count
  FROM pg_stat_activity
  WHERE wait_event_type = 'Lock'
    AND pid <> pg_backend_pid()
),
invalid_indexes AS (
  SELECT
    n.nspname AS schema_name,
    c.relname AS index_name,
    t.relname AS table_name,
    i.indisvalid,
    i.indisready,
    pg_relation_size(c.oid) AS index_bytes
  FROM pg_index i
  JOIN pg_class c ON c.oid = i.indexrelid
  JOIN pg_class t ON t.oid = i.indrelid
  JOIN pg_namespace n ON n.oid = c.relnamespace
  WHERE n.nspname NOT IN ('pg_catalog','information_schema')
    AND (NOT i.indisvalid OR NOT i.indisready)
),
cache_hit AS (
  SELECT
    CASE WHEN (ds.blks_hit + ds.blks_read) = 0 THEN 1.0
         ELSE ds.blks_hit::numeric / (ds.blks_hit + ds.blks_read)::numeric
    END AS hit_ratio
  FROM db_stats ds
),
```

```
dead_tuple_hotspots AS (
  SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    pg_total_relation_size(format('%I.%I', schemaname, relname)) AS total_bytes,
    CASE WHEN (n_live_tup + n_dead_tup) = 0 THEN 0::numeric
         ELSE n_dead_tup::numeric / (n_live_tup + n_dead_tup)::numeric
    END AS dead_ratio,
    last_autovacuum
  FROM pg_stat_user_tables
  WHERE (n_live_tup + n_dead_tup) > 0
),
freeze_age AS (
  SELECT
    max(age(c.relfrozenxid))::bigint AS max_rel_freeze_age
  FROM pg_class c
  JOIN pg_namespace n ON n.oid = c.relnamespace
  WHERE n.nspname NOT IN ('pg_catalog','information_schema')
    AND c.relkind IN ('r','m','t')
),
autovacuum_disabled AS (
  SELECT
    n.nspname AS schema_name,
    c.relname AS table_name,
    c.reloptions
  FROM pg_class c
  JOIN pg_namespace n ON n.oid = c.relnamespace
  WHERE c.relkind IN ('r','m')
    AND n.nspname NOT IN ('pg_catalog','information_schema')
    AND c.reloptions::text LIKE '%autovacuum_enabled=false%'
),
unused_indexes AS (
  SELECT
    s.schemaname,
    s.relname AS table_name,
    s.indexrelname AS index_name,
    s.idx_scan,
    pg_relation_size(format('%I.%I', s.schemaname, s.indexrelname)) AS index_bytes
  FROM pg_stat_user_indexes s
),
repl_primary AS (
  SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    write_lag,
    flush_lag,
    replay_lag
  FROM pg_stat_replication
),
repl_standby AS (
  SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay
),
temp_rate AS (
  SELECT
    ds.temp_bytes,
    ds.stats_reset,
    CASE
      WHEN ds.stats_reset IS NULL THEN NULL::numeric
      WHEN now() <= ds.stats_reset THEN NULL::numeric
      ELSE (ds.temp_bytes::numeric / EXTRACT(epoch FROM (now() - ds.stats_reset))::numeric) * 3600.0
    END AS temp_bytes_per_hour
  FROM db_stats ds
),
checks AS (
  -- BASICS
  SELECT
    b.run_id, b.observed_at,
    'INFO'::text, 'BASICS'::text, 'server_info'::text, 'OK'::text,
```

```
  format('Database=%s | Standby=%s | Uptime=%s', b.dbname, b.is_standby, b.uptime),
  jsonb_build_object('dbname', b.dbname, 'is_standby', b.is_standby, 'uptime', b.uptime, 'server_v
FROM basics b

UNION ALL
-- CAPACITY
SELECT
  b.run_id, b.observed_at,
  'INFO', 'CAPACITY', 'database_size', 'OK',
  format('Database size: %s', pg_size_pretty(ds.db_size_bytes)),
  jsonb_build_object('db_size_bytes', ds.db_size_bytes)
FROM basics b CROSS JOIN db_stats ds

UNION ALL
-- CONNECTION PRESSURE
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN cu.pct_used >= (SELECT conn_critical_pct FROM thresholds) THEN 'CRITICAL'
    WHEN cu.pct_used >= (SELECT conn_warn_pct FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'CONNECTIONS', 'connection_usage',
  CASE WHEN cu.pct_used >= (SELECT conn_warn_pct FROM thresholds) THEN 'ALERT' ELSE 'OK' END,
  format('Connections: %s/%s (%.1f%%)', cu.numbackends, cu.max_connections, cu.pct_used*100.0),
  jsonb_build_object('numbackends', cu.numbackends, 'max_connections', cu.max_connections, 'pct_us
FROM basics b CROSS JOIN conn_usage cu

UNION ALL
-- CACHE HIT
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN ch.hit_ratio < (SELECT cache_hit_critical_below FROM thresholds) THEN 'CRITICAL'
    WHEN ch.hit_ratio < (SELECT cache_hit_warn_below FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'PERFORMANCE', 'cache_hit_ratio',
  CASE WHEN ch.hit_ratio < (SELECT cache_hit_warn_below FROM thresholds) THEN 'ALERT' ELSE 'OK' EN
  format('Cache hit ratio: %.3f', ch.hit_ratio),
  jsonb_build_object('hit_ratio', ch.hit_ratio)
FROM basics b CROSS JOIN cache_hit ch

UNION ALL
-- DEADLOCKS
SELECT
  b.run_id, b.observed_at,
  CASE WHEN ds.deadlocks > 0 THEN 'WARN' ELSE 'INFO' END,
  'LOCKING', 'deadlocks',
  CASE WHEN ds.deadlocks > 0 THEN 'ALERT' ELSE 'OK' END,
  format('Deadlocks since stats reset (%s): %s', ds.stats_reset, ds.deadlocks),
  jsonb_build_object('deadlocks', ds.deadlocks, 'stats_reset', ds.stats_reset)
FROM basics b CROSS JOIN db_stats ds

UNION ALL
-- WAITING LOCK COUNT
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN lc.waiting_count >= (SELECT waiting_locks_critical FROM thresholds) THEN 'CRITICAL'
    WHEN lc.waiting_count >= (SELECT waiting_locks_warn FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'LOCKING', 'waiting_locks_count',
  CASE WHEN lc.waiting_count >= (SELECT waiting_locks_warn FROM thresholds) THEN 'ALERT' ELSE 'OK'
  format('Sessions waiting on locks: %s', lc.waiting_count),
  jsonb_build_object('waiting_count', lc.waiting_count)
FROM basics b CROSS JOIN lock_counts lc

UNION ALL
-- TOP LOCK WAITS
SELECT
```

```
  b.run_id, b.observed_at,
  CASE
    WHEN wl.waiting_for >= (SELECT lock_wait_critical FROM thresholds) THEN 'CRITICAL'
    WHEN wl.waiting_for >= (SELECT lock_wait_warn FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'LOCKING', 'lock_wait_session',
  CASE WHEN wl.waiting_for >= (SELECT lock_wait_warn FROM thresholds) THEN 'ALERT' ELSE 'OK' END,
  format('PID %s waiting %s | user=%s app=%s', wl.pid, wl.waiting_for, wl.usename, wl.application_
  jsonb_build_object('pid', wl.pid, 'usename', wl.usename, 'application_name', wl.application_name
FROM basics b
JOIN (SELECT * FROM waiting_locks ORDER BY waiting_for DESC NULLS LAST LIMIT 20) wl ON true

UNION ALL
-- LONG RUNNING QUERIES
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN al.runtime >= (SELECT long_query_critical FROM thresholds) THEN 'CRITICAL'
    WHEN al.runtime >= (SELECT long_query_warn FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'PERFORMANCE', 'long_running_query',
  CASE WHEN al.runtime >= (SELECT long_query_warn FROM thresholds) THEN 'ALERT' ELSE 'OK' END,
  format('PID %s runtime %s | user=%s app=%s', al.pid, al.runtime, al.usename, al.application_name
  jsonb_build_object('pid', al.pid, 'usename', al.usename, 'application_name', al.application_name
FROM basics b
JOIN (SELECT * FROM activity_long ORDER BY runtime DESC NULLS LAST LIMIT 20) al ON true

UNION ALL
-- INVALID INDEXES
SELECT
  b.run_id, b.observed_at,
  'CRITICAL',
  'INTEGRITY', 'invalid_index',
  'ALERT',
  format('Invalid index %I.%I on %I (%s)', ii.schema_name, ii.index_name, ii.table_name, pg_size_p
  jsonb_build_object('schema', ii.schema_name, 'index', ii.index_name, 'table', ii.table_name, 'in
FROM basics b
JOIN invalid_indexes ii ON true

UNION ALL
-- DEAD TUPLE HOTSPOTS
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN dth.dead_ratio >= (SELECT dead_tuple_ratio_critical FROM thresholds) AND dth.total_bytes
    WHEN dth.dead_ratio >= (SELECT dead_tuple_ratio_warn FROM thresholds)     AND dth.total_bytes
    ELSE 'INFO'
  END,
  'MAINTENANCE', 'dead_tuple_hotspot',
  CASE WHEN dth.dead_ratio >= (SELECT dead_tuple_ratio_warn FROM thresholds) AND dth.total_bytes >
  format('%I.%I dead ratio %.3f | size %s | last_autovacuum=%s', dth.schemaname, dth.relname, dth.
  jsonb_build_object('schema', dth.schemaname, 'table', dth.relname, 'dead_ratio', dth.dead_ratio,
FROM basics b
JOIN (SELECT * FROM dead_tuple_hotspots ORDER BY (dead_ratio * ln(greatest(total_bytes,1))) DESC L

UNION ALL
-- XID WRAPAROUND RISK
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN fa.max_rel_freeze_age >= (s.freeze_max_age * 0.95) THEN 'CRITICAL'
    WHEN fa.max_rel_freeze_age >= (s.freeze_max_age * 0.80) THEN 'WARN'
    ELSE 'INFO'
  END,
  'MAINTENANCE', 'xid_freeze_age',
  CASE WHEN fa.max_rel_freeze_age >= (s.freeze_max_age * 0.80) THEN 'ALERT' ELSE 'OK' END,
  format('Max relfrozenxid age: %s (freeze_max_age=%s)', fa.max_rel_freeze_age, s.freeze_max_age),
  jsonb_build_object('max_rel_freeze_age', fa.max_rel_freeze_age, 'freeze_max_age', s.freeze_max_a
FROM basics b
CROSS JOIN freeze_age fa
```

```
CROSS JOIN settings s

UNION ALL
-- AUTOVACUUM DISABLED
SELECT
  b.run_id, b.observed_at,
  'WARN',
  'MAINTENANCE', 'autovacuum_disabled_table',
  'ALERT',
  format('Autovacuum disabled on %I.%I', av.schema_name, av.table_name),
  jsonb_build_object('schema', av.schema_name, 'table', av.table_name, 'reloptions', av.reloptions
FROM basics b
JOIN autovacuum_disabled av ON true

UNION ALL
-- UNUSED INDEXES
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN ui.idx_scan = 0 AND ui.index_bytes >= (SELECT unused_index_warn_min_bytes FROM thresholds
    WHEN ui.idx_scan = 0 AND ui.index_bytes >= (SELECT unused_index_info_min_bytes FROM thresholds
    ELSE 'INFO'
  END,
  'CAPACITY', 'unused_index',
  CASE WHEN ui.idx_scan = 0 AND ui.index_bytes >= (SELECT unused_index_warn_min_bytes FROM thresho
  format('Unused index %I.%I on %I (idx_scan=%s, size=%s)', ui.schemaname, ui.index_name, ui.table
  jsonb_build_object('schema', ui.schemaname, 'table', ui.table_name, 'index', ui.index_name, 'idx
FROM basics b
JOIN (SELECT * FROM unused_indexes WHERE idx_scan = 0 ORDER BY index_bytes DESC LIMIT 20) ui ON tr

UNION ALL
-- TEMP USAGE RATE
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN tr.temp_bytes_per_hour IS NULL THEN 'INFO'
    WHEN tr.temp_bytes_per_hour >= 20*1024^3 THEN 'CRITICAL'
    WHEN tr.temp_bytes_per_hour >= 5*1024^3 THEN 'WARN'
    ELSE 'INFO'
  END,
  'PERFORMANCE', 'temp_usage_rate',
  CASE WHEN tr.temp_bytes_per_hour IS NOT NULL AND tr.temp_bytes_per_hour >= 5*1024^3 THEN 'ALERT'
  CASE
    WHEN tr.temp_bytes_per_hour IS NULL THEN 'Temp usage rate unavailable (stats_reset is NULL).'
    ELSE format('Temp written per hour (since stats reset %s): %s/hr', tr.stats_reset, pg_size_pre
  END,
  jsonb_build_object('temp_bytes', tr.temp_bytes, 'stats_reset', tr.stats_reset, 'temp_bytes_per_h
FROM basics b
CROSS JOIN temp_rate tr

UNION ALL
-- REPLICATION (PRIMARY)
SELECT
  b.run_id, b.observed_at,
  CASE
    WHEN greatest(coalesce(r.replay_lag, interval '0'), coalesce(r.flush_lag, interval '0'), coale
         >= (SELECT repl_lag_critical FROM thresholds) THEN 'CRITICAL'
    WHEN greatest(coalesce(r.replay_lag, interval '0'), coalesce(r.flush_lag, interval '0'), coale
         >= (SELECT repl_lag_warn FROM thresholds) THEN 'WARN'
    ELSE 'INFO'
  END,
  'REPLICATION', 'primary_replication_lag',
  CASE
    WHEN greatest(coalesce(r.replay_lag, interval '0'), coalesce(r.flush_lag, interval '0'), coale
         >= (SELECT repl_lag_warn FROM thresholds) THEN 'ALERT'
    ELSE 'OK'
  END,
  format('Replica %s (%s) state=%s sync=%s write=%s flush=%s replay=%s',
         r.application_name, coalesce(r.client_addr::text,'?'), r.state, r.sync_state,
         coalesce(r.write_lag::text,'0'), coalesce(r.flush_lag::text,'0'), coalesce(r.replay_lag::
  jsonb_build_object('application_name', r.application_name, 'client_addr', r.client_addr, 'state'
FROM basics b
```

```
      JOIN repl_primary r ON true

      UNION ALL
      -- REPLICATION (STANDBY)
      SELECT
        b.run_id, b.observed_at,
        CASE
          WHEN b.is_standby AND rs.replay_delay IS NOT NULL AND rs.replay_delay >= (SELECT repl_lag_crit
          WHEN b.is_standby AND rs.replay_delay IS NOT NULL AND rs.replay_delay >= (SELECT repl_lag_warn
          ELSE 'INFO'
        END,
        'REPLICATION', 'standby_replay_delay',
        CASE WHEN b.is_standby AND rs.replay_delay IS NOT NULL AND rs.replay_delay >= (SELECT repl_lag_w
        CASE
          WHEN NOT b.is_standby THEN 'Not a standby.'
          WHEN rs.replay_delay IS NULL THEN 'Replay delay unavailable.'
          ELSE format('Standby replay delay: %s', rs.replay_delay)
        END,
        jsonb_build_object('is_standby', b.is_standby, 'replay_delay', rs.replay_delay)
      FROM basics b
      CROSS JOIN repl_standby rs
    )
    SELECT
      run_id, observed_at, severity, category, check_name, status, message, details
    FROM checks
  )
  INSERT INTO public.health_alerts (run_id, observed_at, severity, category, check_name, status, message
  SELECT run_id, observed_at, severity, category, check_name, status, message, details
  FROM report;

  RETURN QUERY
  SELECT
    a.run_id, a.observed_at, a.severity, a.category, a.check_name, a.status, a.message, a.details
  FROM public.health_alerts a
  WHERE a.run_id = v_run_id
  ORDER BY public._health_severity_rank(a.severity) DESC, a.category, a.check_name;

END;
$$;

-- ---------- 4) Summary: latest run rollup ----------
CREATE OR REPLACE FUNCTION public.run_health_summary()
RETURNS TABLE (
  run_id         uuid,
  observed_at    timestamptz,
  critical_count int,
  warn_count     int,
  info_count     int,
  total_count    int,
  has_critical   boolean
)
LANGUAGE sql
AS $$
  WITH latest AS (
    SELECT run_id, max(observed_at) AS observed_at
    FROM public.health_alerts
    GROUP BY run_id
    ORDER BY observed_at DESC
    LIMIT 1
  )
  SELECT
    l.run_id,
    l.observed_at,
    count(*) FILTER (WHERE severity='CRITICAL')::int AS critical_count,
    count(*) FILTER (WHERE severity='WARN')::int     AS warn_count,
    count(*) FILTER (WHERE severity='INFO')::int     AS info_count,
    count(*)::int                                     AS total_count,
    (count(*) FILTER (WHERE severity='CRITICAL') > 0) AS has_critical
  FROM public.health_alerts a
  JOIN latest l ON a.run_id = l.run_id
  GROUP BY l.run_id, l.observed_at;
$$;
```

```
-- ---------- 5) Critical-only: latest run ----------
CREATE OR REPLACE FUNCTION public.run_health_critical()
RETURNS TABLE (
  run_id      uuid,
  observed_at timestamptz,
  severity    text,
  category    text,
  check_name  text,
  status      text,
  message     text,
  details     jsonb
)
LANGUAGE sql
AS $$
  WITH latest AS (
    SELECT run_id, max(observed_at) AS observed_at
    FROM public.health_alerts
    GROUP BY run_id
    ORDER BY observed_at DESC
    LIMIT 1
  )
  SELECT a.run_id, a.observed_at, a.severity, a.category, a.check_name, a.status, a.message, a.details
  FROM public.health_alerts a
  JOIN latest l ON a.run_id = l.run_id
  WHERE a.severity = 'CRITICAL'
  ORDER BY a.category, a.check_name;
$$;

-- ---------- 6) Boolean: any critical alerts in latest run? ----------
CREATE OR REPLACE FUNCTION public.has_critical_alerts()
RETURNS TABLE (
  run_id         uuid,
  observed_at    timestamptz,
  has_critical   boolean,
  critical_count int
)
LANGUAGE sql
AS $$
  SELECT run_id, observed_at, has_critical, critical_count
  FROM public.run_health_summary();
$$;

COMMIT;
```